Pashov Audit Group

# Ostium Security Review

September 14th 2025 - October 11th 2025

# Contents


1. About Pashov Audit Group ..... 3
2. Disclaimer ..... 3
3. Risk Classification ..... 3
4. About Ostium ..... 4
5. Executive Summary ..... 4
6. Findings ..... 5

High findings ..... 8

[H-01] Function `unlockDeposit()` adds discounted assets as traders profit and loss ..... 8
[H-02] Code misses sending rollover fee to liquidity buffer ..... 9
[H-03] LP providers will not get liquidation fee most times ..... 9
[H-04] Code does not scale `accPnlPerToken` in `scaleVariables()` always ..... 10

Medium findings ..... 12

[M-01] `updateAccPnlPerTokenUsed()` only applies positive unrealized PnL ..... 12
[M-02] Orders related to a trade may be executed to replaced trade ..... 13
[M-03] Code does not reset `orderTriggerBlock` when updating trades ..... 13
[M-04] Using average for unrealized PnL settlement gives opportunity to extract value ..... 14
[M-05] Discrepancy for `isLiquidated` in trade and automation callbacks ..... 15
[M-06] `getDynamicTradePriceImpact()` does not use bid/ask as base price ..... 17
[M-07] Attacker can front-run unrealized PnL as they take effect at the end of epoch ..... 18
[M-08] Front-running `unlockDeposit()` can avoid loss distribution ..... 19
[M-09] Deposit/withdraw should not change share price ..... 19
[M-10] Inconsistent price impact calculations ..... 20
[M-11] `depositWithDiscountAndLock()` and `mintWithDiscountAndLock()` lack slippage check ..... 22

Low findings ..... 24

[L-01] `updateAccTotalPnl()` callable only on trades ..... 24
[L-02] Functions `topUpCollateral()` and `removeCollateral()` should round in favor of pool ..... 24
[L-03] Function `updateAccTotalPnl()` only get called if a trade is executed ..... 25
[L-04] Inconsistent maximum group collateral check ..... 25
[L-05] Possible index collision in builder data ..... 25
[L-06] Stale pricestamps can allow backdated executions ..... 28
[L-07] Deposit and mint functions may validate against outdated max supply ..... 29
[L-08] Claiming fees in excess can revert oracle fee refunds ..... 29
[L-09] Allowance bypass in `makeWithdrawRequest()` and `cancelWithdrawRequest()` ..... 30
[L-10] `updateOpenLimitOrder()` may incorrectly alter limit order parameters ..... 30
[L-11] `tryUpdateCurrentMaxSupply()` does not use `maxSupplyIncreaseDailyP` ..... 31
[L-12] Upgrade of `OstiumLockedDepositNft.sol` may block user share claims ..... 31
[L-13] `removePair()` can be DOS by a malicious user ..... 32
[L-14] No `makerFeeP < takerFeeP` validation in `setPairOpeningFees()` ..... 32
[L-15] No zero address check in `registerAuthorizedSigner()` ..... 33
[L-16] Rounding in `mintWithDiscountAndLock()` grants larger discount ..... 33
[L-17] `tryUpdateCurrentMaxSupply` may unexpectedly decrease `currentMaxSupply` ..... 34
[L-18] `performUpkeep` errors if price timestamp exceeds block timestamp ..... 34
[L-19] No prevention in `transfer` / `transferFrom` for withdrawing address ..... 34
[L-20] Updating `oracleFee` might risk DOS in `closeTradeMarketTimeout()` ..... 35
[L-21] Inflation attack in `OstiumVault` ..... 38

# 1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over $100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work here or reach out on Twitter @pashovkrum.

# 2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

# 3. Risk Classification

| Severity | Impact: High | Impact: Medium | Impact: Low |
|---|---|---|---|
| Likelihood: High | Critical | High | Medium |
| Likelihood: Medium | High | Medium | Low |
| Likelihood: Low | Medium | Low | Low |

**Impact**

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

**Likelihood**

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

## 4. About Ostium

Ostium is a decentralized perpetual trading protocol of Real World Assets (RWA). It works across commodities, Forex, cryptocurrencies, and a wide array of long-tail assets.

## 5. Executive Summary

A time-boxed security review of the **0xOstium/smart-contracts** repository was done by Pashov Audit Group, during which **unforgiven**, **Tejas Warambhe**, **IvanFitro**, **aslanbek** engaged to review **Ostium**. A total of **36** issues were uncovered.

| Protocol Summary | |
|---|---|
| **Project Name** | Ostium |
| **Protocol Type** | Perpetual DEX for RWA |
| **Timeline** | September 14th 2025 - October 11th 2025 |

**Review commit hash:**

- 70a6d108efe0313cb532dc0a436f1f736e3e9edf (0xOstium/smart-contracts)

### Scope

`OstiumLockedDepositNft.sol` `OstiumOpenPnl.sol` `OstiumPairInfos.sol` `OstiumPairsStorage.sol` `OstiumPriceRouter.sol` `OstiumPriceUpKeep.sol` `OstiumPrivatePriceUpKeep.sol` `OstiumRegistry.sol` `OstiumTimelockManager.sol` `OstiumTimelockOwner.sol` `OstiumTradesUpKeep.sol` `OstiumTrading.sol` `OstiumTradingCallbacks.sol` `OstiumTradingStorage.sol` `OstiumVault.sol` `OstiumVerifier.sol` `Delegatable.sol` `ChainUtils.sol` `TradingCallbacksLib.sol` `TradingLib.sol` `interfaces/`

# 6. Findings

## Findings count

| Severity | Amount |
|---|---|
| High | 4 |
| Medium | 11 |
| Low | 21 |
| **Total findings** | **36** |

## Summary of findings

| ID | Title | Severity | Status |
|---|---|---|---|
| [H-01] | Function `unlockDeposit()` adds discounted assets as traders profit and loss | High | Acknowledged |
| [H-02] | Code misses sending rollover fee to liquidity buffer | High | Acknowledged |
| [H-03] | LP providers will not get liquidation fee most times | High | Acknowledged |
| [H-04] | Code does not scale `accPnlPerToken` in `scaleVariables()` always | High | Acknowledged |
| [M-01] | `updateAccPnlPerTokenUsed()` only applies positive unrealized PnL | Medium | Acknowledged |
| [M-02] | Orders related to a trade may be executed to replaced trade | Medium | Acknowledged |
| [M-03] | Code does not reset `orderTriggerBlock` when updating trades | Medium | Acknowledged |
| [M-04] | Using average for unrealized PnL settlement gives opportunity to extract value | Medium | Acknowledged |
| [M-05] | Discrepancy for `isLiquidated` in trade and automation callbacks | Medium | Acknowledged |
| [M-06] | `getDynamicTradePriceImpact()` does not use bid/ask as base price | Medium | Acknowledged |
| [M-07] | Attacker can front-run unrealized PnL as they take effect at the end of epoch | Medium | Acknowledged |

| ID | Title | Severity | Status |
|---|---|---|---|
| [M-08] | Front-running `unlockDeposit()` can avoid loss distribution | Medium | Acknowledged |
| [M-09] | Deposit/withdraw should not change share price | Medium | Acknowledged |
| [M-10] | Inconsistent price impact calculations | Medium | Acknowledged |
| [M-11] | `depositWithDiscountAndLock()` and `mintWithDiscountAndLock()` lack slippage check | Medium | Acknowledged |
| [L-01] | `updateAccTotalPnl()` callable only on trades | Low | Acknowledged |
| [L-02] | Functions `topUpCollateral()` and `removeCollateral()` should round in favor of pool | Low | Acknowledged |
| [L-03] | Function `updateAccTotalPnl()` only get called if a trade is executed | Low | Acknowledged |
| [L-04] | Inconsistent maximum group collateral check | Low | Acknowledged |
| [L-05] | Possible index collision in builder data | Low | Acknowledged |
| [L-06] | Stale pricestamps can allow backdated executions | Low | Acknowledged |
| [L-07] | Deposit and mint functions may validate against outdated max supply | Low | Acknowledged |
| [L-08] | Claiming fees in excess can revert oracle fee refunds | Low | Acknowledged |
| [L-09] | Allowance bypass in `makeWithdrawRequest()` and `cancelWithdrawRequest()` | Low | Acknowledged |
| [L-10] | `updateOpenLimitOrder()` may incorrectly alter limit order parameters | Low | Acknowledged |
| [L-11] | `tryUpdateCurrentMaxSupply()` does not use `maxSupplyIncreaseDailyP` | Low | Acknowledged |
| [L-12] | Upgrade of `OstiumLockedDepositNft.sol` may block user share claims | Low | Acknowledged |
| [L-13] | `removePair()` can be DOS by a malicious user | Low | Acknowledged |
| [L-14] | No `makerFeeP < takerFeeP` validation in `setPairOpeningFees()` | Low | Acknowledged |

| ID | Title | Severity | Status |
|---|---|---|---|
| [L-15] | No zero address check in `registerAuthorizedSigner()` | Low | Acknowledged |
| [L-16] | Rounding in `mintWithDiscountAndLock()` grants larger discount | Low | Acknowledged |
| [L-17] | `tryUpdateCurrentMaxSupply` may unexpectedly decrease `currentMaxSupply` | Low | Acknowledged |
| [L-18] | `performUpkeep` errors if price timestamp exceeds block timestamp | Low | Acknowledged |
| [L-19] | No prevention in `transfer` / `transferFrom` for withdrawing address | Low | Acknowledged |
| [L-20] | Updating `oracleFee` might risk DOS in `closeTradeMarketTimeout()` | Low | Acknowledged |
| [L-21] | Inflation attack in `OstiumVault` | Low | Acknowledged |

# High findings

## [H-01] Function `unlockDeposit()` adds discounted assets as traders profit and loss

### Severity

**Impact:** Medium

**Likelihood:** High

### Description

Users can deposit their funds with a lock and receive extra shares. Later when those locks are expire, code adds the discounted assets to the variable `accPnlPerToken` and updates the share price to socialize the discounted assets.

```
        int256 accPnlDelta = d.assetsDiscount.mulDiv(PRECISION_18, totalSupply(),
Math.Rounding.Ceil).toInt256();

        accPnlPerToken += accPnlDelta;
        if (accPnlPerToken > maxAccPnlPerToken().toInt256()) {
            revert NotEnoughAssets();
        }

        lockedDepositNft.burn(depositId);

        accPnlPerTokenUsed += accPnlDelta;
        updateShareToAssetsPrice();
```

This creates multiple issues: 1. Variable `accPnlPerToken` is used to track traders profit, and loss and code uses net profit and loss to calculate share price if only the accumulated value is positive. Adding discounted assets to `accPnlPerToken` interferes with this accumulated net profit and loss of traders, which works as liquidity buffer. 2. Changing variable `accPnlPerToken` doesn't change share price when `accPnlPerToken < 0`. It means adding discounted deposits to `accPnlPerToken` won't change share price in those cases, and a as result vault would become insolvent because total supply includes the discounted assets, but share price doesn't reflect that, and `supply * price` would be higher than total assets.

### Recommendations

Distribute discounted assets by decreasing the variable `accRewardsPerToken`.

## [H-02] Code misses sending rollover fee to liquidity buffer

### Severity

**Impact:** Medium

**Likelihood:** High

### Description

According to the doc, the rollover fee should be accumulated in the liquidity buffer, and it should be used to settle traders profit and loss. But in the current code, code keeps the rollover fee as dev fee in the storage contract. In function `getTradeValue()` code calculates the rollover fee and in function `getTradeValuePure()` code reduces the rollover fee from trade value and later code sends the `tradeValue` to the user in the `unregisterTrade()` function and rollover fee stays in the storage contract. As a result code calculates the rollover fee correctly, but doesn't send those tokens to the liquidity buffer. Those fees stays in the storage contract, and won't be used as a buffer to settle the traders profit.

As a liquidity buffer is implemented in the vault with `accPnlPerToken` variable, code should send the rollover fee to the vault with function `receiveAssets()` so that those fees would be used as a liquidity buffer to settle the traders profit and loss.

### Recommendations

Send the rollover to the vault with function `receiveAssets()`.

## [H-03] LP providers will not get liquidation fee most times

### Severity

**Impact:** High

**Likelihood:** Medium

### Description

When liquidation happens, code use function `receiveAssets()` to send funds to the vault:

```
        // 3 USDC vault reward
        if (liquidationFee > 0) {
            storageT.transferUsdc(address(storageT), address(this), liquidationFee);
            vault.receiveAssets(liquidationFee, trade.trader);
            emit VaultLiqFeeCharged(orderId, tradeId, trade.trader, liquidationFee);
        }
```

The issue is that: 1- funds received by `receiveAssets()` don't affect the share price immediately, and some users won't receive those rewards if they interact with the vault before the rewards distribution finalization. 2- funds received by `receiveAssets()` is accumulated with variable `accPnlPerToken` as traders loss, and code doesn't consider this accumulated trader losses for share calculation if `accPnlPerToken < 0` in function `updateShareToAssetsPrice()`.

As a result of #2 if traders net profit was negative, then the liquidation fee won't affect the share price, unlike what's described in the documentation.

### Recommendations

Use function `distributeReward()` to distribute the liquidation fee.

## [H-04] Code does not scale `accPnlPerToken` in `scaleVariables()` always

### Severity

**Impact:** Medium

**Likelihood:** High

### Description

In function `scaleVariables()` code scales the value of `accPnlPerToken` to reflect the changed balance:

```
        uint256 supply = totalSupply();

        if (accPnlPerToken < 0) {
            accPnlPerToken = accPnlPerToken * supply.toInt256()
                / (isDeposit ? (supply + shares).toInt256() : (supply - shares).toInt256());
        }
    }
```

The issue is that when `accPnlPerToken > 0`, code doesn't update the value, and as a result the implied accumulated PnL would be wrong. For example: 1. Suppose there are 100 token and 100 shares in the vault, and traders profit is 10 token so `accPnlPerToken` would be 0.1, and share price would be 0.9. 2. User A deposits 90 token and would receive 100 shares, and code won't update `accPnlPerToken` because it's value is positive. 3. Now total share would be 200, and `accPnlPerToken` would be 0.1, and the implied accumulated PnL would be `200 * 0.1 = 20`, which wrong.

The impact is that code would use the wrong value for PnL, and the share price and yield calculations would be wrong over time.

### Recommendations

Always scale the value of the `accPnlPerToken` in the `scaleVariables()` functions.

# Medium findings

## [M-01] `updateAccPnlPerTokenUsed()` only applies positive unrealized PnL

### Severity

**Impact:** Medium

**Likelihood:** Medium

### Description

When an epoch ends and a new epoch starts, the function `updateAccPnlPerTokenUsed()` is executed, and it updates the share price. This function is called by function `startNewEpoch()`, which sends unrealized accumulated profit, and loss at the start of the epoch and at the end of the epoch as input for function `updateAccPnlPerTokenUsed()`:

```
uint256 currentEpochPositiveOpenPnl = vault.currentEpochPositiveOpenPnl();

uint256 finalNewEpochPositiveOpenPnl = vault.updateAccPnlPerTokenUsed(
    currentEpochPositiveOpenPnl, newEpochOpenPnl > 0 ? uint256(newEpochOpenPnl) : 0
);
```

The issue is that this function only sends the positive unrealized PnL for the vault, and vault only applies the positive traders PnL in the vault's share price. It means if traders have unrealized losses, then vault share price won't be updated even after the epoch ends. This would create situations in which users can extract value from the vault. For example: 1. Traders made realized profits, and the vault of `accPnlPerToken` is positive, and the share price decreased (if traders make a realized loss, then the share price will increase). 2. UserA has a trade that has a big unrealized loss, and the epochs net unrealized PnL is negative. 3. In function `startNewEpoch()`, because `newEpochOpenPnl < 0` code won't send the unrealized loss to the vault, and the vault share price won't be increased. 4. To capture some of the value that LPs receive from the loss, UserA would deposit a large amount of assets into the vault (for example, owning 50% of vault's share) and then close his trade and make a realized loss. 5. Now the code would increase the vault's share price, and 50% of the gained yield will be received by UserA himself. 6. Then UserA would withdraw his funds.

UserA can create a big delta neutral position to perform this attack too. As a result of this issue, attacker have opportunities to extract value from the vault when there's a position with loss and the accumulated PnL in the vault is positive.

### Recommendations

Always update the vault's share price based on accumulated unrealized PnL.

# [M-02] Orders related to a trade may be executed to replaced trade

## Severity

**Impact:** Medium

**Likelihood:** Medium

## Description

When a trade is executed, code finds an empty index in the `openTrades[][][]` variable and stores the new trade. There could be a pending order for an open trade. Those orders contain the trades position index, and when those orders executed, they will be are executed for that index.

```
        IOstiumTradingStorage.Trade memory t = storageT.getOpenTrade(sender, pairIndex, index);
...
        storageT.storePendingRemoveCollateral(
            IOstiumTradingStorage.PendingRemoveCollateral(removeAmount, sender, pairIndex,
index), orderId
        );
```

The issue is that when a trade is closed and replaced with another trade, the related orders that are related to trade's index aren't closed, and they may remain in the contract's state and executed later for the replaced trade. Different types of orders can be created for a trade, like take profit, stop loss, close, and remove collateral. Before executing those orders, the trade can be liquidated, and another trade can be replaced with that trade's index in the `openTrades[_trader][_pairIndex][_index]` variable and later the order can be executed for the new trade, witch isn't the intent of the trader. This is one of the scenarios: 1- UserA has a trade in index 2 and wants to close 50% of their position, and creates an order to close 50% of the trade in index 2. 2- The price of assets changes, and UserA's trade gets liquidated. 3- UserA opens another trade, and it's stored in index 2. 4- Now the open order for closing 50% of the trade can be executed for the new trade, but it wasn't user intention to execute this order on the new trade.

## Recommendations

Have a unique trade ID and store it in the trade and order, and when executing the order, check that those IDs are the same and the order belongs to that trade.

# [M-03] Code does not reset `orderTriggerBlock` when updating trades

## Severity

**Impact:** Medium

**Likelihood:** Medium

### Description

Code use the variable `orderTriggerBlock` to track the events triggered for an order or trade. To perform an operation on an order or trade, code checks that there's no current ongoing trigger by calling `checkNoPendingTrigger()` function:

```
    function checkNoPendingTrigger(
        IOstiumTradingStorage storageT,
        address trader,
        uint16 pairIndex,
        uint8 index,
        IOstiumTradingStorage.LimitOrder orderType,
        uint256 triggerTimeout
    ) public view returns (bool) {
        uint256 triggerBlock = storageT.orderTriggerBlock(trader, pairIndex, index, orderType);

        if (triggerBlock == 0 || (triggerBlock > 0 && ChainUtils.getBlockNumber() -
triggerBlock >= triggerTimeout)) {
            return true;
        }
        return false;
    }
```

The issue is that when removing a trade, code doesn't reset the value of the `orderTriggerBlock[]` for that specific the index. And code may reuse index to store the future trades. As a result, the next trade will have value in `orderTriggerBlock` and some operations will fail for that trade. For example: 1- UserA has an open trade and calls `removeCollateral`, and code creates a remove collateral trigger. 2- Next, user trade is liquidated, but the value of `orderTriggerBlock[]` isn't removed for that specific trade's index. 3- UserA opens another trade, and it's stored in the same trade index position. 4- Now the new trade has the value of `orderTriggerBlock[]` for removing collateral, and it won't be possible to perform that action for some time.

This issue can happen with different triggers, and it would create failure if users try to do high frequency trading.

### Recommendations

In function `unregisterTrade()`, reset all the trigger types value for that trade in variable `orderTriggerBlock`

## [M-04] Using average for unrealized PnL settlement gives opportunity to extract value

### Severity

**Impact:** Low

**Likelihood:** High

### Description

When an epoch ends, code settles the unrealized profit and loss in the vault. It uses the average of the snapshot of the unrealized PnL during the epoch:

```
        int256 newEpochOpenPnl =
            nextEpochValues.length >= requestsCount ? average(nextEpochValues) :
currentEpochPositiveOpenPnl.toInt256();

        uint256 finalNewEpochPositiveOpenPnl = vault.updateAccPnlPerTokenUsed(
            currentEpochPositiveOpenPnl, newEpochOpenPnl > 0 ? uint256(newEpochOpenPnl) : 0
        );
```

The issue is that there may be a high difference between the current net PnL and its average value, specially if the price changes dramatically near the end of the epoch. As a result, in the next epoch, users would have the opportunity to deposit/withdraw while this gain/loss isn't included in the vault's share price and benefit more than other LP providers.

### Recommendations

Don't use average or settle the deposits/withdraw with the share price of next epoch.

## [M-05] Discrepancy for `isLiquidated` in trade and automation callbacks

### Severity

**Impact:** Medium

**Likelihood:** Medium

### Description

According to the docs, the liquidation trigger should be calculated based on the market price and executed with price after impact. The `closeTradeMarketCallback()` follows this behavior:

```
                (
                    TradingCallbacksLib.TradeValueResult memory tvResult,
                    TradingCallbacksLib.PriceImpactResult memory piResult
                ) = TradingCallbacksLib.getTradeAndPriceData(
                    a,
                    t,
                    pairInfos,
                    i.initialLeverage,

IOstiumPairsStorage(registry.getContractAddress('pairsStorage')).pairMaxLeverage(t.pairIndex),
```

```
                    collateralToClose,
                    true
                );

                bool isLiquidated = tvResult.tradeValue < tvResult.liqMarginValue;
```

But in functions `executeAutomationCloseOrderCallback()` code recalculates the `isLiquidated` based on the trade value of price after impact:

```
                if (isMarketPrice) {
                    (tvResult.profitP,) = TradingCallbacksLib.currentPercentProfit(
                        t.openPrice.toInt256(),
                        piResult.priceAfterImpact.toInt256(),
                        t.buy,
                        int32(t.leverage),
                        int32(i.initialLeverage)
                    );
                    tvResult.tradeValue = pairInfos.getTradeValuePure(
                        t.collateral, tvResult.profitP, tvResult.rolloverFees,
tvResult.fundingFees
                    );

                    isLiquidated = tvResult.tradeValue < tvResult.liqMarginValue;
                }
```

As a result, if user has SL close to the liquidation price in order to avoid the liquidation, if the SL gets triggered, then because of this second `isLiquidated` calculation based on the `priceAfterImpact`, the position will be liquidated. Also this happens in `getHandleRemoveCollateralCancelReason()` function too:

```
        (int256 profitP, int256 maxPnlP) = currentPercentProfit(
            trade.openPrice.toInt256(),
            result.priceAfterImpact.toInt256(),
            trade.buy,
            int32(trade.leverage),
            int32(initialLeverage)
        );

        uint32 maxLeverage = pairsStorage.pairMaxLeverage(trade.pairIndex);
        (uint256 tradeValue, uint256 liqMarginValue,,) = pairInfos.getTradeValue(
            trade.trader,
            trade.pairIndex,
            trade.index,
            trade.buy,
            trade.collateral,
            trade.leverage,
            profitP,
            maxLeverage
        );

        bool isLiquidated = tradeValue < liqMarginValue;
        uint256 usdcSentToTrader = isLiquidated ? 0 : tradeValue;
```

Code calculates `isLiquidated` based on the `priceAfterImpact`.

This is a discrepancy between the docs and implementation, and will cause user to pay the liquidation fee while their position is healthy according to the market price. Also, if this should be the default behavior, then users can avoid this liquidation by closing their orders with the market price when the position is about to liquidate.

### Recommendations

Either fix the `closeTradeMarketCallback()` to trigger liquidation based on the price after impact, or change the `executeAutomationCloseOrderCallback()` and `getHandleRemoveCollateralCancelReason()` to not trigger liquidation with market price.

## [M-06] `getDynamicTradePriceImpact()` does not use bid/ask as base price

### Severity

**Impact:** Low

**Likelihood:** High

### Description

Function `getDynamicTradePriceImpact()` is supposed to calculate the trade price using the dynamic price impact method:

```
        priceAfterImpact = uint192(price);

        priceImpactP = _priceImpactFunction(
            netVolThreshold,
            priceImpactK,
            trade.buy,
            isOpen,
            tradeNotional,
            initialImbalance,
            uint192(price),
            uint192(ask),
            uint192(bid)
        );

        // Apply price impact to base price
        if (priceImpactP > 0) {
            if (isOpen == trade.buy) {
                priceAfterImpact = priceAfterImpact * (PRECISION_18 + (priceImpactP / 100)) /
PRECISION_18;
            } else {
                priceAfterImpact =
                    priceImpactP < 100e18 ? priceAfterImpact * (PRECISION_18 - (priceImpactP /
100)) / PRECISION_18 : 0;
            }
        }
```

Code use `uint192(price)` as the base price and then calculates the price impact percentage and updates the value of `priceImpactP`. The issue is that the code uses `a.price` as the base price and doesn't take bid/ask price into consideration. For example, if the price impact percentage was zero, the code would return `a.price`, which is the current middle price, and code won't take into account the big/ask price. As a result, in some scenarios, the buy and sell price would be the same regardless of the bid/ask price.

### Recommendations

Use this for base price like in `_getTradePriceImpact()`:

```
        bool aboveSpot = (isOpen == isLong);
        int192 usedPrice = aboveSpot ? ask : bid;
```

## [M-07] Attacker can front-run unrealized PnL as they take effect at the end of epoch

### Severity

**Impact:** Medium

**Likelihood:** Medium

### Description

Code calculates and apply unrealized profit/loss at the start/end of the epoch in function `startNewEpoch()`:

```
        uint256 currentEpochPositiveOpenPnl = vault.currentEpochPositiveOpenPnl();

        int256 newEpochOpenPnl =
            nextEpochValues.length >= requestsCount ? average(nextEpochValues) :
currentEpochPositiveOpenPnl.toInt256();

        uint256 finalNewEpochPositiveOpenPnl = vault.updateAccPnlPerTokenUsed(
            currentEpochPositiveOpenPnl, newEpochOpenPnl > 0 ? uint256(newEpochOpenPnl) : 0
        );
```

The issue is that this will allow an attacker to front-run the unrealized profits and deposit before the epoch ends or front-run the unrealized losses and withdraw before the epoch ends (Also the attacker or users need to have pending withdrawal requests to do this).

### Recommendations

Apply unrealized loss in share price in each trade and deposit/withdraw or perform deposit/withdraw with share price as minimum of the end of the epoch and request time (This will change deposit/withdraw process).

## [M-08] Front-running `unlockDeposit()` can avoid loss distribution

### Severity

**Impact:** Medium

**Likelihood:** Medium

### Description

When users deposits token with a lock, the code gives them bonus shares, and after the lock expires, the bonus is socialized among the share token holders in function `unlockDeposit()`:

```
        int256 accPnlDelta = d.assetsDiscount.mulDiv(PRECISION_18, totalSupply(),
Math.Rounding.Ceil).toInt256();

        accPnlPerToken += accPnlDelta;
        if (accPnlPerToken > maxAccPnlPerToken().toInt256()) {
            revert NotEnoughAssets();
        }

        lockedDepositNft.burn(depositId);

        accPnlPerTokenUsed += accPnlDelta;
        updateShareToAssetsPrice();
```

The issue is that code distributes this loss suddenly and it's unpredictable. Attacker can predict the unlock time or just front-run the unlock transaction and perform the withdraw before the unlock to avoid this loss of socialization.

### Recommendations

Either socialize the bonus/loss overtime in each epoch or consider those pending bonuses for asset calculations when users perform withdraw.

## [M-09] Deposit/withdraw should not change share price

### Severity

**Impact:** Low

**Likelihood:** High

### Description

In ERC4626, deposit and withdraw operation shouldn't change the share price, otherwise, if one user deposits or withdraw tokens, it would effect the other users. Right now, in the current implementation, the share price is calculated based on `accPnlPerToken`, which is also responsible for keeping the accumulated PnL. This creates contradicted situation: 1. During deposit/withdraw the `accPnlPerToken` should be scaled to reflect the correct accumulated PnL. 2. During deposit/withdraw the share price shouldn't be changed, and so `accPnlPerToken` shouldn't be changed.

The issue is that in the current code, instead of tracking the absolute value of accumulated PnL, code keeps track of accumulated PnL per share token and uses it for both purposes mentioned above. In the current code, this doesn't lead to a bug because: 1. When `accPnlPerToken > 0`, code doesn't scale its value inside function `scaleVariables()`, which is another bug, and if it's fixed the it would result in a share price change during deposit withdraw. 2. When `accPnlPerToken <0`, code scale its value, but because of a design choice, when `accPnlPerToken <0`, then code doesn't consider `accPnlPerToken` value for share price calculation.

So as you can see, there's a bug here, but because of another bug and a design choice, this bugs effect is canceled, but if the other bug is fixed or the design choice is changed, then the bug will show itself.

### Recommendations

Keep track of the accumulated PnL instead of the `accPnlPerToken` and perform the division by total share inside the `updateShareToAssetsPrice()` function.

## [M-10] Inconsistent price impact calculations

### Severity

**Impact:** Low

**Likelihood:** High

### Description

The `TradingCallbacksLib::getDynamicTradePriceImpact` helps in calculating the final price impact before registering the trade. It is calculated before the `OstiumTradingCallbacks::registerTrade()` call in `OstiumTradingCallbacks::openTradeMarketCallback()` and `OstiumTradingCallbacks::executeAutomationOpenOrderCallback()`.

However, the collateral value used to calculate the `priceAfterImpact` and `priceImpactP` is before fee deductions, but the dynamic spread is updated using the post-fee collateral:

```
    function openTradeMarketCallback(IOstiumPriceUpKeep.PriceUpKeepAnswer calldata a) external
notDone {

        // . . .
        if (a.price <= 0 || a.bid <= 0 || a.ask <= 0) {
            cancelReason = CancelReason.MARKET_CLOSED;
        } else if (isDayTradeClosed(trade.pairIndex, trade.leverage, a.isDayTradingClosed)) {
            cancelReason = CancelReason.DAY_TRADE_NOT_ALLOWED;
        } else {
            result = TradingCallbacksLib.getDynamicTradePriceImpact(
                a.price, int192(a.ask), int192(a.bid), true, trade, pairInfos, trade.collateral
            );

            trade.openPrice = result.priceAfterImpact.toUint192();               <<@

            cancelReason = TradingCallbacksLib.getOpenTradeMarketCancelReason(
                isPaused,
                wantedPrice,
                slippageP,
                uint192(a.price),
                trade,
                result.priceImpactP,
                IOstiumPairInfos(registry.getContractAddress('pairInfos')),
                pairsStorage,
                storageT
            );
        }

        if (cancelReason == CancelReason.NONE) {
            trade = registerTrade(a.orderId, trade, uint192(a.price), bf);              <<@
-- // trade.collateral is stored post Fee deductions

            if (result.isDynamic) {
                _updateDynamicSpreadVolumes(
                    trade.pairIndex, true, trade.buy, trade.collateral, trade.leverage,
pairInfos                 <<@
                );
            }
       // . . .
```

A similar flow can be seen in `executeAutomationOpenOrderCallback()`. From this observation, we can infer: 1. Price impact / entry price is a bit overstated and will keep changing as fee parameters are changed. 2. Inconsistent builder fees will affect the price impact, as it is not a fixed percentage and will differ for each user. 3. Subsequent pricing will be less accurate.

## Recommendations

It is recommended to calculate the price impact after deducting fees from the trade's collateral.

## [M-11] `depositWithDiscountAndLock()` and `mintWithDiscountAndLock()` lack slippage check

### Severity

**Impact:** Medium

**Likelihood:** Medium

### Description

`depositWithDiscountAndLock()` and `mintWithDiscountAndLock()` are used to acquire shares at a discount. Both functions have the same issue, but `depositWithDiscountAndLock()` will be used.

```
function depositWithDiscountAndLock(uint256 assets, uint32 lockDuration, address receiver)
        external
        checks(assets)
        validDiscount(lockDuration)
        returns (uint256)
    {
        uint256 simulatedAssets = assets
            * (PRECISION_18 * uint256(100) + lockDiscountP(collateralizationP(), lockDuration))
            / (PRECISION_18 * uint256(100));

        if (simulatedAssets > maxDeposit(receiver)) {
            revert AboveMaxDeposit();
        }

        return _executeDiscountAndLock(simulatedAssets, assets,
previewDeposit(simulatedAssets), lockDuration, receiver);
    }
```

`simulatedAssets` are the inflated assets that apply the discount from `lockDiscountP()`. Users deposit assets but receive an equivalent amount of shares as `simulatedAssets`, which are inflated to account for the discount. Now, let's examine `lockDiscountP()`:

```
function lockDiscountP(uint256 collatP, uint32 lockDuration) public view returns (uint256) {
        return (
            collatP <= uint16(100) * PRECISION_2
                ? uint256(maxDiscountP) * 1e16
                : collatP <= maxDiscountThresholdP
                    ? uint256(maxDiscountP) * 1e16 * (maxDiscountThresholdP - collatP)
                        / (maxDiscountThresholdP - uint16(100) * PRECISION_2)
                    : 0
        ) * lockDuration / MAX_LOCK_DURATION;
    }
```

If `collatP` is sufficiently high, meaning the protocol has high collateralization, the applied discount is reduced. This creates a problem because a user can lock their assets for a period but receive only a minimal bonus, which does not fairly compensate for the time the shares are locked, making it unfair to the user.

To better illustrate the issue, consider the following example:

1. Bob calls `depositWithDiscountAndLock()` with `amount = 1e18` and `lockDuration = MAX_LOCK_DURATION` (365 days).
2. `lockDiscountP()` calculates the discount, which is 1% because of the high `collatP`.
3. `_executeDiscountAndLock()` is executed, transferring the assets to the contract and locking the shares for 1 year.
4. Bob has locked his shares for 1 year but receives only a 1% bonus, which is insufficient. Now he must wait for the shares to unlock, and given such a small bonus, he would have preferred to simply call `deposit()`.

## Recommendations

To solve the problem, implement a slippage check on the obtained discount. If the discount is lower than the user-specified minimum, the transaction should revert.

# Low findings

## [L-01] `updateAccTotalPnl()` callable only on trades

During the epoch when a trade is executed, code calls `updateAccTotalPnl()` to update the current accumulated PnL, and later those accumulated PnL is used to record the PnL during the epoch, and use it at the end of the epoch to calculate unrealized profit and loss. The issue is that in the current code, function `newOpenPnlRequestOrEpoch()` is only called when a trade is executed and if there's no trade but there's price change, then the accumulated PnL won't be updated for that no-trade interval, and as a result, the recorded net PnL would be wrong for those times. Off-chain component should be able to trigger `updateAccTotalPnl()` without needing a trade execution.

## [L-02] Functions `topUpCollateral()` and `removeCollateral()` should round in favor of pool

Function `topUpCollateral()` and `removeCollateral()` change the collateral and leverage values. Because of rounding error, the position's trade size will be changed too:

```
uint256 tradeSize = t.collateral.mulDiv(t.leverage, 100, Math.Rounding.Ceil);
uint256 newCollateral = t.collateral - removeAmount;
uint32 newLeverage = (tradeSize * PRECISION_6 / newCollateral / 1e4).toUint32();

if (tradeSize * PRECISION_6 % (newCollateral * 1e4) != 0) {
    newCollateral = tradeSize * 1e2 / newLeverage;

    if (newCollateral < t.collateral) {
        removeAmount = t.collateral - newCollateral;
    } else {
        revert WrongParams();
    }
}
```

The issue is that when the trade size is changed, the absolute value of the profit and loss also changes. Therefore, if the user's position is in profit, increasing the trade size would benefit the user. Conversely, if the user's position is in loss, decreasing the trade size would also benefit the user. Code doesn't check the profit or loss of the user, and always rounds in the same direction. So user can add or remove a small wei of collateral, and change the profit a little bit, and repeat the action. Because of oracle fee, and bounded leverage the attack vector is limited right now, but it's may be possible to combine the attack vector with other functionalities to exploit the code in the future.

## [L-03] Function `updateAccTotalPnl()` only get called if a trade is executed

Function `updateAccTotalPnl()` updates the unrealized PnL value and it's important for unrealized PnL calculation at the end of the epoch. The issue is that code only calls `updateAccTotalPnl()` when a trade is executed, and it's not possible to call it directly or via off-chain work. So if there was no trade for some time, then code won't update the unrealized PnL, and the total PnL calculation wouldn't have the correct value at the end of the epoch.

## [L-04] Inconsistent maximum group collateral check

The `OstiumTrading::openTrade()` function allows users to open trades, which are later fulfilled by the keeper. The `TradingCallbacksLib::withinExposureLimits()` does not allow the total collateral to surpass the group's maximum collateral, which is checked for before registering the trade:

```
    function withinExposureLimits(
        uint16 pairIndex,
        bool buy,
        uint256 collateral,
        uint32 leverage,
        uint256 price,
        IOstiumPairsStorage pairsStorage,
        IOstiumTradingStorage tradingStorage
    ) public view returns (bool) {
        return tradingStorage.openInterest(pairIndex, buy ? 0 : 1) * price / PRECISION_18 / 1e12
            + collateral * leverage / 100 <= tradingStorage.openInterest(pairIndex, 2)
            && pairsStorage.groupCollateral(pairIndex, buy) + collateral <=
pairsStorage.groupMaxCollateral(pairIndex);                <<@
    }
```

However, the collateral used here includes builder, opening, dev, vault, and oracle fees, but the actual group collateral update excludes all such fees:

```
        // function registerTrade()
        pairsStorage.updateGroupCollateral(trade.pairIndex, trade.collateral, trade.buy, true);
```

This could lead to unintended trade reverts at the max collateral boundary.

## [L-05] Possible index collision in builder data

The `OstiumTrading::openTrade()` allows users to open market or limit orders along with builder data. The builder data is stored with respect to the index at which the order is created. If the order type is limit, the index is determined via the first available empty index:

```
        if (orderType != IOstiumTradingStorage.OpenOrderType.MARKET) {
            uint8 index = storageT.firstEmptyOpenLimitIndex(sender, t.pairIndex);         <<@

            uint32 currTimestamp = block.timestamp.toUint32();
```

```
            storageT.storeOpenLimitOrder(
                IOstiumTradingStorage.OpenLimitOrder(
                    t.collateral,
                    t.openPrice,
                    t.tp,
                    t.sl,
                    sender,
                    t.leverage,
                    currTimestamp,
                    currTimestamp,
                    t.pairIndex,
                    orderType,
                    index,                    <<@
                    t.buy
                ),
                bf
            );
```

Similarly, for market orders, the index is determined as the `orderId` derived from `currentOrderId` from the price router:

```
        } else {
            uint256 orderId =
IOstiumPriceRouter(registry.getContractAddress('priceRouter')).getPrice(            <<@
                t.pairIndex, IOstiumPriceUpKeep.OrderType.MARKET_OPEN, block.timestamp
            );

            storageT.storePendingMarketOrder(
                IOstiumTradingStorage.PendingMarketOrderV2(
                    0,
                    t.openPrice,
                    slippageP.toUint32(),
                    IOstiumTradingStorage.Trade(t.collateral, 0, t.tp, t.sl, sender,
t.leverage, t.pairIndex, 0, t.buy),
                    0
                ),
                orderId,                      <<@
                true,
                bf
            );

            emit MarketOpenOrderInitiated(orderId, sender, t.pairIndex);
        }
```

For market orders, the `orderId` would start from 0 and keep incrementing, whereas the `index` for non-market orders would keep on fluctuating from 0 till max allowed orders for a particular user.

Hence, there's a chance that a user placing multiple orders could have a builder index collision, which would override the builder data, leading to loss of fees for the builder or unintended fee transfer to the other builder.

Proof of Concept (PoC):-

1. A user creates a limit order, which assigns `orderId` as `1` due to the first current available open limit index. The builder data is now stored with this `orderId`.
2. User proceeds to create a market order with different builder data, which assigns `orderId` as `1`, as the `currentOrderId` starts from 0.
3. The market order created here would overwrite the builder data at index `1`.

Add the following test case inside `OstiumTradingStorage.t.sol`:

```
    function test_BuilderFee_Collision_LimitThenMarket() public {
        // Prepare distinct builder fees and builders
        IOstiumTradingStorage.BuilderFee memory bfLimit =
            IOstiumTradingStorage.BuilderFee(address(0xB1), 1_000); // 0.001%
        IOstiumTradingStorage.BuilderFee memory bfMarket =
            IOstiumTradingStorage.BuilderFee(address(0xB2), 2_000); // 0.002%

        // Creating a limit order at index 1
        IOstiumTradingStorage.OpenLimitOrder memory o = IOstiumTradingStorage.OpenLimitOrder({
            collateral: t.collateral,
            targetPrice: t.openPrice,
            tp: t.tp,
            sl: t.sl,
            trader: DEFAULT_SENDER,
            leverage: t.leverage,
            createdAt: 0,
            lastUpdated: 0,
            pairIndex: t.pairIndex,
            orderType: IOstiumTradingStorage.OpenOrderType.LIMIT,
            index: 1, // collide with market orderId 1
            buy: t.buy
        });

        // Store orders in storage
        vm.prank(address(mockTrading));
        tradingStorage.storeOpenLimitOrder(o, bfLimit);

        // Builder fee set for index 1
        {
            IOstiumTradingStorage.BuilderFee memory stored =
                tradingStorage.getBuilderData(DEFAULT_SENDER, t.pairIndex, 1);
            assertEq(stored.builder, bfLimit.builder);
            assertEq(stored.builderFee, bfLimit.builderFee);
        }

        // Now, creating a market open pending order with orderId = 1
        IOstiumTradingStorage.PendingMarketOrderV2 memory mo =
IOstiumTradingStorage.PendingMarketOrderV2({
            block: 0,
            wantedPrice: t.openPrice,
            slippageP: 500, // arbitrary
            trade: t,
            percentage: 0
        });
```

```
        vm.prank(address(mockTrading));
        tradingStorage.storePendingMarketOrder(mo, 1, true, bfMarket);

        // Collision: builderData for key 1 now holds market builder fee, overwriting the limit
one
        {
            IOstiumTradingStorage.BuilderFee memory afterFee =
                tradingStorage.getBuilderData(DEFAULT_SENDER, t.pairIndex, 1);
            assertEq(afterFee.builder, bfMarket.builder);
            assertEq(afterFee.builderFee, bfMarket.builderFee);
        }
    }
```

A similar case can be observed if we create the market order before the limit order.

It is recommended to ensure that `currentOrderId` always starts above the maximum allowed open limit orders.

## [L-06] Stale pricestamps can allow backdated executions

The `OstiumTrading::executeAutomationOrder()` allows the keeper to execute automated orders via `OstiumTradesUpKeep::performUpkeep()` and set up pending triggers, respectively. The function checks for the `priceTimestamp` to be at least up to date with the order's `createdAt` timestamp:

```
    function executeAutomationOrder(
        IOstiumTradingStorage.LimitOrder orderType,
        address trader,
        uint16 pairIndex,
        uint8 index,
        uint256 priceTimestamp
    ) external onlyTradesUpKeep notDone pairIndexListed(pairIndex) returns
(IOstiumTrading.AutomationOrderStatus) {
        // . . .
        if (orderType == IOstiumTradingStorage.LimitOrder.OPEN) {
            if (!storageT.hasOpenLimitOrder(trader, pairIndex, index)) {
                return IOstiumTrading.AutomationOrderStatus.NO_LIMIT;
            }
            isNotPaused();

            IOstiumTradingStorage.OpenLimitOrder memory openOrder =
storageT.getOpenLimitOrder(trader, pairIndex, index);
            if (priceTimestamp < openOrder.createdAt) {                                <<@
                return IOstiumTrading.AutomationOrderStatus.BACKDATED_EXECUTION;
            }
        } else {
        // . . .
```

However, the `OstiumTrading::updateOpenLimitOrder()` allows users to update the target price, TP, and SL for the order. The intent of the user behind such an update can be considered to be equivalent to a new order creation.

Hence, it is recommended to consider the `lastUpdated` to check for `BACKDATED_EXECUTION` instead of `createdAt` in the `executeAutomationOrder()` call.

## [L-07] Deposit and mint functions may validate against outdated max supply

The `OstiumVault::tryUpdateCurrentMaxSupply()` can be called publicly and is persistently called inside `sendAssets()`, `receiveAssets()`, and `updateAccPnlPerTokenUsed()` to ensure a mint cap is maintained:

```
    function tryUpdateCurrentMaxSupply() public {
        if (block.timestamp - lastMaxSupplyUpdateTs >= 24 hours) {
            currentMaxSupply =
                totalSupply() * (uint16(100) * PRECISION_2 + maxSupplyIncreaseDailyP) /
(PRECISION_2 * uint16(100));
            lastMaxSupplyUpdateTs = uint32(block.timestamp);

            emit CurrentMaxSupplyUpdated(currentMaxSupply);
        }
    }
```

However, the `deposit()`, `mint()`, and `mintWithDiscountAndLock()` fail to update the supply cap, which can lead to a higher max supply than intended or revert a mint/deposit transaction due to stale max current supply.

It is recommended to add `tryUpdateCurrentMaxSupply()` to calls above to update the max supply before checking against the `maxMint()`.

## [L-08] Claiming fees in excess can revert oracle fee refunds

The collected fees can be claimed via governance using `OstiumTradingStorage::claimFees()`:

```
    function claimFees(uint256 _amount) external onlyGov {
        uint256 _devFees = devFees;
        if (_amount > _devFees || _amount == 0) {
            revert WrongParams();
        }
        devFees -= _amount;

        SafeERC20.safeTransfer(IERC20(usdc), registry.dev(), _amount);
    }
```

The `devFees` variable stores all the oracle fees stored when orders are opened. Similarly, the `refundOracleFee()` is used for reimbursing the oracle fees back to the trader.

However, there's no check to ensure that the lien `devFees` is sufficient to fulfill the refunds of all the orders that might get closed in the future.

Hence, in a case where total potential refunds post claimed fees exceed the `devFees` balance, the `OstiumTradingCallbacks::closeTradeMarketCallback()` and `OstiumTrading::closeTradeMarketTimeout()` functions will revert.

It is recommended to only claim fees that are in excess and leave enough funds for potential refunds.

## [L-09] Allowance bypass in `makeWithdrawRequest()` and `cancelWithdrawRequest()`

The `OstiumVault::makeWithdrawRequest()` and `OstiumVault::cancelWithdrawRequest()` allow users and addresses with allowances for a particular user to create or cancel a withdrawal request.

The allowance is checked for the given shares:

```
        if (sender != owner && (allowance == 0 || allowance < shares)) {
            revert NotAllowed(sender);
        }
```

However, this allowance is not reduced or accounted for, which allows the address with allowance to actually create or cancel withdrawals by calling the function multiple times.

This scenario can be replicated as follows:-

1. A user provides allowance (X) smaller than the actual share holdings (Y) to an address `A` (which can probably be an external integration or a strategy-based contract).
2. Even though address `A` has a lesser allowance than the share holdings, they can spam `makeWithdrawRequest()` and `cancelWithdrawRequest()` till they inflate or deflate `totalSharesBeingWithdrawn()` respectively.

This opens up a griefing attack vector that can be leveraged to time grief withdrawals by canceling at the end of every epoch or simply grief `transfer()` / `transferFrom()` functions that utilize `totalSharesBeingWithdrawn()`, potentially blocking transfers unless explicit allowance is set back to 0.

It is recommended to separate actual share allowance concerns from the create/cancel withdrawal allowances; otherwise, document this behavior.

## [L-10] `updateOpenLimitOrder()` may incorrectly alter limit order parameters

Users can use `updateOpenLimitOrder()` to update their open limit orders parameters. This functions accept the open order index in the `openLimitOrderIds[][][]` variable:

```
    function updateOpenLimitOrder(uint16 pairIndex, uint8 index, uint192 price, uint192 tp,
uint192 sl)
        external
        notDone
    {
```

The open limit orders can be replaced with other limit orders when they are closed. The same pattern is applied to trades too. Users update and perform operations on trades with function that require trade's index as input, and trades can be replaced by other trades. The issue is that between the time user signs the transaction and the transaction is executed, the index can be replaced with another trade or order, and as a result code would execute the operation on the wrong trade/order. This would be an impactful issue for users who perform high frequency trading. Trades and orders should have a unique ID that is included in the transaction and is checked in function to make sure operations are executed for the correct trade or order and not the newly replaced one.

## [L-11] `tryUpdateCurrentMaxSupply()` does not use `maxSupplyIncreaseDailyP`

Function `tryUpdateCurrentMaxSupply()` is supposed to update the max supply based on the percentage per day:

```
        if (block.timestamp - lastMaxSupplyUpdateTs >= 24 hours) {
            currentMaxSupply =
                totalSupply() * (uint16(100) * PRECISION_2 + maxSupplyIncreaseDailyP) /
(PRECISION_2 * uint16(100));
            lastMaxSupplyUpdateTs = uint32(block.timestamp);

            emit CurrentMaxSupplyUpdated(currentMaxSupply);
        }
```

The issue is that code only applies the change when `tryUpdateCurrentMaxSupply()` is called and sets the value of the `lastMaxSupplyUpdateTs` for the current timestamp. So if this function isn't called for two days (directly or via other deposit/withdraw functions), then daily increase would be applied for two days. To fix this code should consider the passed time for increased amount calculation or increase the `lastMaxSupplyUpdateTs` by 24 hours instead of setting it to the current timestamp.

## [L-12] Upgrade of `OstiumLockedDepositNft.sol` may block user share claims

The `unlockDeposit()` allows users to claim the shares associated with their locked deposits once the locking period has elapsed. However, the function retrieves the `lockedDepositNft` contract from the registry:

```
IOstiumLockedDepositNft lockedDepositNft =
    IOstiumLockedDepositNft(registry.getContractAddress("lockedDepositNft"));
```

The issue arises if the registry has been updated to point to a new `lockedDepositNft` contract. In that case, some users will be unable to claim their shares because the `ownerOf()` check will revert, and the corresponding `depositId` no longer exists in the new contract. As a result, users are effectively blocked from withdrawing their shares.

Recommendation: Introduce a mechanism to retrieve and interact with the old NFT contract. This would allow users to successfully claim their shares, even after the registry has been updated.

## [L-13] `removePair()` can be DOS by a malicious user

`removePair()` is used to remove a pair so it can no longer be used to open a trade.

```
function removePair(uint16 _pairIndex) external onlyGov pairListed(_pairIndex) {
    if
(IOstiumTradingStorage(registry.getContractAddress('tradingStorage')).pairTradersCount(_pairIndex)
> 0) {
        revert PairNotEmpty();
    }

    Pair memory p = pairs[_pairIndex];

    isPairListed[p.from][p.to] = false;
    isPairIndexListed[_pairIndex] = false;

    emit PairRemoved(_pairIndex, p.from, p.to);
}
```

Before removing a pair, the function checks that there are no active trades. A malicious user could open a trade with very high collateral and an extreme take profit solely to block the pair from being removed, effectively creating DOS.

Recommendation: Implement a `forceRemovePair()` function that removes the pair without checking for active trades. This avoids the DOS risk while still allowing governance to unlist pairs when necessary.

## [L-14] No `makerFeeP < takerFeeP` validation in `setPairOpeningFees()`

When a user opens a trade, `maxOpeningFee` is calculated using `takerFeeP`, assuming it represents the worst-case scenario (i.e., the maximum fee the user could pay). It does not use `makerFeeP` because the logic assumes `takerFeeP > makerFeeP` by default.

The issue is that this assumption isn't enforced in `setPairOpeningFees()`:

```
function setPairOpeningFees(uint16 pairIndex, PairOpeningFees calldata value) public onlyGov {
    if (
        value.makerFeeP > MAX_FEEP || value.takerFeeP > MAX_FEEP || value.usageFeeP > MAX_FEEP
            || value.utilizationThresholdP >= MAX_USAGE_THRESHOLDP || value.makerMaxLeverage >
MAX_MAKER_LEVERAGE
```

```
            || value.vaultFeePercent > 100
    ) {
        revert WrongParams();
    }
    pairOpeningFees[pairIndex] = value;

    emit PairOpeningFeesUpdated(pairIndex, value);
}
```

This allows `makerFeeP > takerFeeP`, which breaks the assumption and means `maxOpeningFee` might not be calculated using the true worst-case fee.

Recommendations: * If the design requires `makerFeeP < takerFeeP`, enforce it in `setPairOpeningFees()`. * If not, update `openTrade()` to compare both fees and use the higher value when calculating `maxOpeningFee`.

## [L-15] No zero address check in `registerAuthorizedSigner()`

The `registerAuthorizedSigner()` function is used to register valid signers. The issue is that it does not check whether `signerAddress` is equal to `address(0)`:

```
function registerAuthorizedSigner(address signerAddress) public onlyGov {
    if (isAuthorizedSigner[signerAddress]) revert AlreadyAuthorizedSigner(signerAddress);
    isAuthorizedSigner[signerAddress] = true;
    emit AuthorizedSignerAdded(signerAddress);
}
```

If `address(0)` is mistakenly registered as a valid signer, it will cause all invalid signatures in `verify()` to be valid. This happens because `ecrecover()` does not revert on invalid signatures but instead returns `address(0)`.

Recommendation: Add a validation check in `registerAuthorizedSigner()` to revert if `signerAddress == address(0)`.

## [L-16] Rounding in `mintWithDiscountAndLock()` grants larger discount

The `mintWithDiscountAndLock()` function allows users to obtain shares at a discounted rate. The issue arises because the calculation of `assets` rounds Ceil:

```
function previewMint(uint256 shares) public view virtual returns (uint256) {
        return _convertToAssets(shares, Math.Rounding.Ceil);
    }
```

Whereas the calculation of `assetsDeposited` rounds down:

```
assets * (PRECISION_18 * uint256(100))
       / (PRECISION_18 * uint256(100) + lockDiscountP(collateralizationP(), lockDuration))
```

This mismatch can result in the protocol granting users a slightly larger discount than intended when computing `assetsDiscount`.

Recommendation: Apply rounding Ceil to the `assetsDeposited` calculation to ensure the discount behaves as expected.

## [L-17] `tryUpdateCurrentMaxSupply` may unexpectedly decrease `currentMaxSupply`

In tryUpdateCurrentMaxSupply, if the totalSupply has decreased substantially, currentMaxSupply will decrease as well.

If this is not intended, consider adding a condition to check if the new max supply is greater than currentMaxSupply, and update currentMaxSupply only in that case.

## [L-18] `performUpkeep` errors if price timestamp exceeds block timestamp

Per Arbitrum docs, it is possible for the block.timestamp to be as far as 24 hours in the past, or 1 hour in the future. If price timestamp is a few seconds ago, and block.timestamp is a few minutes ago, this will lead to panic in OstiumPriceRouter#getPrice, which is used in closeTradeMarket, removeCollateral, executeAutomationOrder, openTrade.

```
    function getPrice(uint16 pairIndex, IOstiumPriceUpKeep.OrderType orderType, uint256
timestamp)
        external
        onlyTrading
        returns (uint256)
    {
        if (block.timestamp - timestamp > maxTsValidity) {
            revert WrongTimestamp();
        }
```

Consider checking if timestamp > block.timestamp.

## [L-19] No prevention in `transfer` / `transferFrom` for withdrawing address

```
   // Override ERC-20 functions (prevent sending to an address that is withdrawing)
    function transfer(address to, uint256 amount) public override(ERC20Upgradeable, IERC20)
```

If the comment is outdated, consider removing it. Otherwise, consider updating the implementation to check that the recipient does not have a pending withdrawal.

## [L-20] Updating `oracleFee` might risk DOS in `closeTradeMarketTimeout()`

`closeTradeMarketTimeout()` is used to close a non-executed close trade once `marketOrdersTimeout` is reached.

```
function closeTradeMarketTimeout(uint256 _order, bool retry) external notDone {
        address sender = _msgSender();
        IOstiumTradingStorage storageT =
IOstiumTradingStorage(registry.getContractAddress('tradingStorage'));

        (
            uint256 _block,
            uint192 wantedPrice,
            uint32 slippageP,
            IOstiumTradingStorage.Trade memory trade,
            uint16 percentage
        ) = storageT.reqID_pendingMarketOrder(_order);

        if (trade.trader == address(0)) {
            revert NoTradeToTimeoutFound(_order);
        }

        if (trade.trader != sender) {
            revert NotYourOrder(_order, trade.trader);
        }

        if (trade.leverage > 0) {
            revert NotCloseMarketTimeoutOrder(_order);
        }

        if (_block == 0 || ChainUtils.getBlockNumber() < _block + marketOrdersTimeout) {
            revert WaitTimeout(_order);
        }

        storageT.unregisterPendingMarketOrder(_order, false);

        uint256 tradeId = storageT.getOpenTradeInfo(sender, trade.pairIndex,
trade.index).tradeId;

        if (retry) {
            (bool success,) = address(this).delegatecall(
                abi.encodeWithSignature(
                    'closeTradeMarket(uint16,uint8,uint16,uint192,uint32)',
                    trade.pairIndex,
                    trade.index,
                    percentage,
                    wantedPrice,
                    slippageP
                )
            );
            //@audit-ok if success will continue, needs a return here, oracleFee will be
returned always -> is refunded for the previous order
            if (!success) {
                emit MarketCloseFailed(tradeId, sender, trade.pairIndex);
```

```
            }
        }
        // Always refund oracle fee regardless of partial or full close
@>      uint256 oracleFee =

IOstiumPairsStorage(registry.getContractAddress('pairsStorage')).pairOracleFee(trade.pairIndex);
        storageT.refundOracleFee(oracleFee);
        storageT.transferUsdc(address(storageT), sender, oracleFee);
        emit OracleFeeRefunded(tradeId, sender, trade.pairIndex, oracleFee);

        emit MarketCloseTimeoutExecutedV2(
            _order,
            tradeId,
            IOstiumTradingStorage.PendingMarketOrderV2({
                trade: trade,
                block: _block,
                wantedPrice: wantedPrice,
                slippageP: slippageP,
                percentage: percentage
            })
        );
    }
```

The `oracleFee` is refunded to the user because the close trade was never executed.

The issue is that this `oracleFee` is retrieved from `pairsStorage`, which can be updated. As a result, while the trade is pending closure, if the `oracleFee` changes, the user may receive a higher or lower refund than intended, also causing incorrect `devFee` accounting. This causes three problems:

1. Allows users to claim fees meant for the protocol team if `oracleFee` is increased, effectively stealing them.
2. Users receive less `oracleFee` than intended if it is decreased.
3. Allow users to claim the `oracleFee` of other users and potentially create a DOS situation, as `refundOracleFee()` could revert with `RefundOracleFeeFailed()`.

```
function refundOracleFee(uint256 _amount) external onlyTrading {
        if (_amount > devFees) {
            revert RefundOracleFeeFailed();
        }
        devFees -= _amount;
    }
```

This means users cannot claim their tokens until another user contributes with `oracleFee`, `openingFees` or `rolloverFees` to `devFees`.

To better demonstrate the issue, copy the following POC into `OstiumTrading.t.sol`.

```
function test_IncreaseOracleFee_closeTradeMarketTimeout() public {

        //Bob order
        uint256 orderId = 2;
        mockTradingStorage.storeTrade(
```

```
            t,
            IOstiumTradingStorage.TradeInfo(
                orderId, t.collateral * 1e12 * t.leverage / 100 * PRECISION_18 / t.openPrice,
t.leverage, 0, 0, 0, false
            )
        );
        mockTradingStorage.storePendingMarketOrder(
            IOstiumTradingStorage.PendingMarketOrderV2(1, closeT.openPrice, 15, closeT, 100e2),
orderId, false, bf
        );

        //Alice order
        uint256 orderId2 = 3;
        mockTradingStorage.storeTrade(
            t,
            IOstiumTradingStorage.TradeInfo(
                orderId2, t.collateral * 1e12 * t.leverage / 100 * PRECISION_18 / t.openPrice,
t.leverage, 0, 0, 0, false
            )
        );
        mockTradingStorage.storePendingMarketOrder(
            IOstiumTradingStorage.PendingMarketOrderV2(1, closeT.openPrice, 15, closeT, 100e2),
orderId2, false, bf
        );

        vm.prank(GOV_WALLET);
        usdc.mint(address(mockTradingStorage), 100);

        vm.prank(GOV_WALLET);
        usdc.mint(address(mockTradingStorage), 100);

        vm.roll(MARKET_ORDERS_TIMEOUT + 1);

        //increase OracleFee, initial oracleFee: 100
        vm.prank(GOV_WALLET);
        fee = IOstiumPairsStorage.Fee({name: bytes32('crypto'), liqFeeP: 50, oracleFee: 200,
minLevPos: 75e6});
        pairStorage.updateFee(0,fee);


        vm.expectEmit(true, true, false, true, address(trading));
        emit MarketCloseTimeoutExecutedV2(
            orderId,
            mockTradingStorage.getOpenTradeInfo(DEFAULT_SENDER, 0, 0).tradeId,
            IOstiumTradingStorage.PendingMarketOrderV2(1, closeT.openPrice, 15, closeT, 100e2)
        );
        vm.prank(DEFAULT_SENDER);
        trading.closeTradeMarketTimeout(orderId, false);

        assertEq(mockTradingStorage.devFees(), 0);

        //Alice can't call closeTradeMarketTimeout() because revert
        vm.prank(DEFAULT_SENDER);

vm.expectRevert("ERC20InsufficientBalance(0x5991A2dF15A8F6A256D3Ec51E99254Cd3fb576A9, 0, 200)");
        trading.closeTradeMarketTimeout(orderId2, false);
    }
```

As shown, Bob pays 100 in `oracleFee` but receives 200 after it is updated. This prevents Alice from subtracting her amount because Bob has already received her `oracleFee`, and Bob ends up receiving more tokens than intended.

> Note: The same issue also occurs in `closeTradeMarketCallback()`.

**Recommendations**

To solve the problem, add a new parameter in `PendingMarketOrderV2` to store the `oracleFee` the user pays in `closeTradeMarket()`, and then use this stored value in `closeTradeMarketTimeout()`.

```
struct PendingMarketOrderV2 {
        uint256 block;
        uint192 wantedPrice; // PRECISION_18
        uint32 slippageP; // PRECISION_2 (%)
        Trade trade;
        uint16 percentage; // PRECISION_2 (%)
+       uint256 oraleFeePaid;
    }
```

## [L-21] Inflation attack in `OstiumVault`

If the vault is ever redeployed, since OZ functions with virtual shares are overridden, it would be vulnerable to the inflation attack (via distributeReward), where the attacker would set the price to a value that will lead to significant precision loss for future depositors.

**Recommendations**

Ensure the first deposit is executed atomically at deployment.